International Journal of Forecasting 41 (2025) 1296–1309

Contents lists available at ScienceDirect

International Journal of Forecasting

journal homepage: www.elsevier.com/locate/ijforecast

A projected nonlinear state-space model for forecasting time series signals

Christian Donner [a,1], Anuj Mishra [b,1], Hideaki Shimazaki [b,c,*]

[a] Swiss Data Science Center, Zurich, Switzerland
[b] Graduate School of Informatics, Kyoto University, Kyoto, Japan
[c] Center for Human Nature, Artificial Intelligence, and Neuroscience (CHAIN), Hokkaido University, Sapporo, Japan

ARTICLE INFO

Keywords:
State space model
Filtering/smoothing
Nonlinear transition
Ridge kernel
Chaotic attractors

ABSTRACT

Learning and forecasting stochastic time series is essential in various scientific fields. However, despite the proposals of nonlinear filters and deep-learning methods, it remains challenging to capture nonlinear dynamics from a few noisy samples and predict future trajectories with uncertainty estimates while maintaining computational efficiency. Here, we propose a fast algorithm to learn and forecast nonlinear dynamics from noisy time series data. A key feature of the proposed model is kernel functions applied to projected lines, enabling the fast and efficient capture of nonlinearities in the latent dynamics. Through empirical case studies and benchmarking, the model demonstrates its effectiveness at learning and forecasting complex nonlinear dynamics, offering a valuable tool for researchers and practitioners in time series analysis.

© 2025 The Author(s). Published by Elsevier B.V. on behalf of International Institute of Forecasters. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

Learning and forecasting stochastic nonlinear dynamics from noisy observations is imperative in studies involving time series analysis. Examples of time series are the gain returns of stock exchanges or simultaneous activities of multiple neurons to analyze cognitive dynamics. While a wide range of solutions, from the extensions of the classical Kalman filters to modern deep-learning methods, have been proposed, it remains challenging to provide fast and reliable predictions for nonlinear systems when the data are noisy and scarce.

In this study, we approach this challenge by integrating the low-rank dynamics assumption (Thibeault, Allard, & Desrosiers, 2024) and suitable regularization for nonlinearity in a state-space modeling framework. We introduce a fast inference and learning algorithm for nonlinear transitions in latent dynamics using univariate Gaussian kernel functions on projected lines in the latent space. Unlike popular kernels such as the Gaussian radial basis function (RBF) and squared exponential (SE) kernel, we restrict our kernels to be the ridge functions (Seshadri, Yuchi, & Parks, 2019) to simplify distance calculations in the multidimensional latent space by measuring it along a one-dimensional projection. This way, one can significantly accelerate inference and learning compared to the RBF and SE kernel while potentially capturing the low-rank structure of the dynamics. Our results demonstrate that the proposed model outperforms deep-learning alternatives in predicting chaotic dynamics from a few noisy data samples. Moreover, we showcase that the method more reliably reconstructs the vector fields of the latent nonlinear dynamics than deep-learning methods, indicating superior generalization to the dynamics starting from previously unseen points.

The paper is organized as follows. In Section 2, we discuss related approaches to the problem at hand. In

* Corresponding author at: Graduate School of Informatics, Kyoto University, Kyoto, Japan.
E-mail addresses: chdonner@ethz.ch (C. Donner), h.shimazaki@i.kyoto-u.ac.jp (H. Shimazaki).

[1] These authors contributed equally.

https://doi.org/10.1016/j.ijforecast.2025.01.002
0169-2070/© 2025 The Author(s). Published by Elsevier B.V. on behalf of International Institute of Forecasters. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Section 3, we introduce our proposed state-space model, derive filtering and smoothing equations, and present an efficient training algorithm for this model. In Section 4, we show its forecasting performance on some classical nonlinear systems and its computational efficiency, and then do an exhaustive comparison to a wide range of forecasting methods, from classical to recently peer-reviewed state-of-the-art models, in predicting the time series of more than 100 chaotic systems. Furthermore, we compare the methods' capability of reconstructing underlying dynamical systems from noisy data.

2. Related work

In time series modeling, the classical approach for estimating linear systems is the Kalman filter (Chen et al., 2003; Kalman, 1960), providing the exact posterior for the linear system with Gaussian noise. For other systems, we generally need to approximate the posterior, either analytically or via sampling. The extended Kalman filter (EKF) (Durbin & Koopman, 2012) and the unscented Kalman filter (UKF) (Julier & Uhlmann, 2004) are classic examples of the analytic and deterministic sampling-based approaches, respectively. These methods are limited in that they can only provide approximate filter equations when the dynamics' parametric model is known. To overcome the limitation of the parametric models, researchers made considerable efforts to model the dynamics in a nonparametric manner, often using kernel functions or Gaussian processes (GPs). Ghahramani and Roweis (1999) first used kernel functions to model the observation and state models, employed the EKF for a linearized model, and further proposed the expectation–maximization (EM) algorithm to train the model. Ko and Fox (2011), Ko, Klein, Fox, and Haehnel (2007) constructed GP-EKF and GP-UKF as extensions of EKF and UKF, respectively, by modeling the observation and transition using GPs. Wang, Fleet, and Hertzmann (2005, 2007) constructed the GP dynamical model (GP-DM) that found maximum a posteriori (MAP) estimates of latent states and hyperparameters. To obtain the full posterior distributions, Deisenroth, Huber, and Hanebeck (2009) utilized an analytic method called moment matching to construct prediction and filter densities for nonlinear observations and transitions modeled by GPs that had the squared exponential kernels as covariance functions. For this model, Turner, Deisenroth, and Rasmussen (2010) employed the moment matching method for inference of filter and smoothing density, and developed an EM algorithm for learning the model parameters. Since then, researchers have proposed other types of posterior approximation methods, such as particle filters (Frigola, Lindsten, Schön, & Rasmussen, 2013), expectation propagation (Deisenroth & Mohamed, 2012), variational Bayes (Frigola, Chen, & Rasmussen, 2014), and variational schemes utilizing neural networks (Krishnan, Shalit, & Sontag, 2015, 2017; Zhao & Park, 2020).

Another view of the non-parametric approaches above is to classify them by their form of kernel functions. While the classical approaches used a combination of RBFs (Ghahramani & Roweis, 1999; Wang et al., 2005, 2007; Zhao & Park, 2020) to represent the nonlinearity, many of the extended methods used the more general SE kernels (Deisenroth et al., 2009; Deisenroth & Mohamed, 2012; Duncker, Bohner, Boussard, & Sahani, 2019; Turner et al., 2010). These kernel functions provide a distance metric in the potentially high-dimensional dynamical space. As we discuss below, both choices suffer from computational complexity, while the SE kernel is more expressive at describing the nonlinearity than the RBF.

Deep-learning techniques based on artificial neural networks became popular as alternative approaches in time series prediction in recent years. These models include the classical employment of recurrent networks (Honkela & Valpola, 2004; Pearlmutter, 1989; Valpola & Karhunen, 2002) and deep-learning techniques (e.g., long short-term memory (LSTM) (Hochreiter & Schmidhuber, 1997; Salinas, Flunkert, Gasthaus, & Januschowski, 2020), transformers (d'Ascoli, Becker, Mathis, Schwaller, & Kilbertus, 2023; Kim, Kim, Tae, Park, Choi, & Choo, 2021; Liu et al., 2023; Vaswani et al., 2017), and NBEATS (Oreshkin, Carpov, Chapados, & Bengio, 2020)). Recurrent neural networks, best suited for time series or sequence-based data, learn the patterns of sequences by having recurrent connections: the network units are connected to themselves with a time delay. RNNs are popular but difficult to train because the gradients can disappear or explode when learning long-term dependencies (Bengio, Simard, & Frasconi, 1994). The LSTM model was one of the first techniques that solved the vanishing gradient problem (Hochreiter & Schmidhuber, 1997). Even though new techniques were added, such as better activations, residual layers, batch normalization, dropout, adaptive learning rate schedules, etc., the core architectures stayed the same until the mid-2010s. However, nowadays, the transformer architecture (Vaswani et al., 2017) is considered the preferred model for most sequential modeling tasks. Unlike RNNs that use recurrent connections for information flow over time, transformers use a self-attention mechanism (Bahdanau, Cho, & Bengio, 2014) to capture global dependencies across the whole sequence directly. Hence, transformers demonstrate unprecedented performance in sequence-to-sequence tasks. They can model long-term dependencies effectively, avoid vanishing gradient issues common in RNNs, and allow for parallel computation during training. Therefore, transformers have become a promising choice for forecasting tasks (d'Ascoli et al., 2023; Kim et al., 2021; Liu et al., 2023). Separately from these recurrence- and attention-based models, a new model has been introduced recently, namely NBEATS (Oreshkin et al., 2020). With impressive forecasting ability, NBEATS uses a simple but powerful architecture of ensemble feed-forward networks with stacked residual blocks of forecasts and backcasts. Unlike probabilistic approaches to modeling time series, most deep-learning methods do not consider uncertainties in their dynamics and are sometimes limited to forecasting a one-dimensional signal only by default. Nevertheless, we compare our model with these competing architectures and report the competitive forecasting performance of the proposed model.

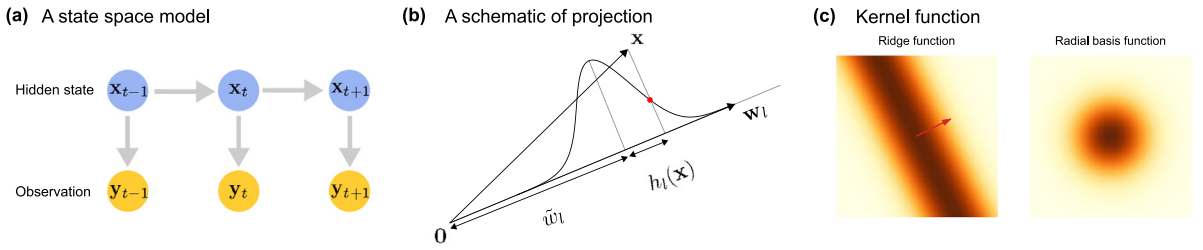


Fig. 1. A schematic illustration of the projected nonlinear state-space model. **(a)** The state-space model is composed of hidden states and observed variables. **(b)** A schematic illustration of the projection operation, $\phi_{\mathrm{nl},l}(\mathbf{x})$, where the direction of the projection is given by $\mathbf{w}_l$. The illustration assumes that $\mathbf{w}_l$ is a unit vector. **(c)** The ridge function in two-dimensional latent space (left). For comparison, the RBF kernel is shown on the right.

3. Model specification

A state-space model is composed of an observation model and a state model (Fig. 1**a**). The former determines the generative process of the observations $\mathbf{y}_t$ given the latent variables $\mathbf{x}_t$, while the latter describes the discrete-time dynamics of the latent variable $\mathbf{x}_t$. Here, $\mathbf{y}_t \in \mathbb{R}^{D_\mathbf{y}}$ and $\mathbf{x}_t \in \mathbb{R}^{D_\mathbf{x}}$ are the observable and state variables at discrete time t ($t = 1, 2, \ldots, T$).

In this study, we assume that the state model is discrete in time, stochastic, and nonlinear:

$$\mathbf{x}_t = \mathbf{f}(\mathbf{x}_{t-1}) + \boldsymbol{\xi}_t, \tag{1}$$

where $\mathbf{f} : \mathbb{R}^{D_\mathbf{x}} \to \mathbb{R}^{D_\mathbf{x}}$ is a set of $D_\mathbf{x}$ nonlinear state transition functions, $\mathbf{f}(\mathbf{x}) = [f_1(\mathbf{x}), \ldots, f_{D_\mathbf{x}}(\mathbf{x})]^\top$. $\boldsymbol{\xi}_t$ represents the latent state noise. We assume zero-mean Gaussian noise, i.e., $\boldsymbol{\xi}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_x)$, where $\boldsymbol{\Sigma}_x$ is a $D_\mathbf{x} \times D_\mathbf{x}$ covariance matrix.

We model the nonlinear state transition using kernel functions as follows:

$$\mathbf{f}(\mathbf{x}_{t-1}) = \begin{pmatrix}\mathbf{A}_{\mathrm{lin}} & \mathbf{A}_{\mathrm{nl}}\end{pmatrix} \begin{pmatrix}\boldsymbol{\phi}_{\mathrm{lin}}(\mathbf{x}_{t-1}) \\ \boldsymbol{\phi}_{\mathrm{nl}}(\mathbf{x}_{t-1})\end{pmatrix} + \mathbf{b} = \mathbf{A}\boldsymbol{\phi}(\mathbf{x}_{t-1}) + \mathbf{b}. \tag{2}$$

The feature vector $\boldsymbol{\phi}(\mathbf{x})$ is composed of linear features $\boldsymbol{\phi}_{\mathrm{lin}}(\mathbf{x}) = \mathbf{x}$ and nonlinear features $\boldsymbol{\phi}_{\mathrm{nl}}(\mathbf{x})$, which are subsequently specified using kernels. Note that in the case $\mathbf{A}_{\mathrm{nl}} = 0$, the standard linear state-space model is recovered. Hence, the nonlinear part extends and compensates for deviations from the linear model. For the nonlinear feature vector $\boldsymbol{\phi}_{\mathrm{nl}}(\mathbf{x}) = [\phi_{\mathrm{nl},1}(\mathbf{x}), \ldots, \phi_{\mathrm{nl},L}(\mathbf{x})]^\top$, we choose a linear projection of the latent variables passed through a squared exponential nonlinearity. That is,

$$\phi_{\mathrm{nl},l}(\mathbf{x}) = \exp\left(-\frac{h_l(\mathbf{x})^2}{2}\right). \tag{3}$$

The input $h_l(\mathbf{x})$ is given as a projection of the hidden states to the weight vector $\mathbf{w}_l$ (Fig. 1**b**):

$$h_l(\mathbf{x}) = \mathbf{w}_l^\top \mathbf{x} - \tilde{w}_l, \tag{4}$$

for $l = 1, \ldots, L$. Here, $\mathbf{w}_l$ is a $D_\mathbf{x} \times 1$ weight vector and $\tilde{w}_l$ is a scalar constant. Note that $\mathbf{w}_l^\top \mathbf{x}$ is a projection of $\mathbf{x}$ to the vector $\mathbf{w}_l$, and $\tilde{w}_l$ forms an offset (inducing) point of the lth kernel in this direction (Fig. 1**b**). Each kernel adds $d = D_\mathbf{x} + 1$ parameters to the model. Fig. 1**c** depicts the resulting ridge function in two-dimensional latent space. For a comparison, the RBF is shown on the right panel. Note that the kernel function employed in this study is concentrated only along the projected line and non-concentrated in the orthogonal directions. In contrast, the RBF is concentrated in all dimensions.

The introduced dynamic model can be seen from different perspectives. First, in the light of neural network terminology, it can be interpreted as a residual layer (He, Zhang, Ren, & Sun, 2016). For the nonlinear part, the input $\mathbf{x}_{t-1}$ is transformed by a linear layer, where $\mathbf{w}_l$ and $\tilde{w}_l$ are weights and biases, respectively. Then, the linearly transformed input is passed through a non-linearity, i.e., the squared exponential function, and then again linearly weighted by $\mathbf{A}_{\mathrm{nl}}$. The linear part can be seen as a skip connection which bypasses the non-linearity and is only linearly transformed.

A second viewpoint of the functional choice in Eq. (2) is in the context of dynamical systems. The dynamics are modeled with a (presumably dominant) linear part, and a nonlinear part models the remaining higher-order terms that correct an otherwise linear model. These higher-order terms appear naturally even in simple systems when the dynamics are modeled in a low-dimensional manifold (Thibeault et al., 2024). Since, in general, the system dimensionality is not known, it is important to account for these non-linearities.

While more complex choices are possible, we assume only a linear observation model in this work:

$$\mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{d} + \boldsymbol{\zeta}_t, \tag{5}$$

where the noise is again Gaussian, i.e., $\boldsymbol{\zeta}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_y)$. Hereafter, we call the proposed model a projected nonlinear state-space (PNL-SS) model.

For an observed dataset $\mathbf{y}_{1:T}$, the model above implies the likelihood

$$p(\mathbf{y}_{1:T}|\vartheta) = \int d\mathbf{x}_{0:T}\, p(\mathbf{x}_0|\vartheta) \prod_{t=1}^{T} p(\mathbf{y}_t|\mathbf{x}_t, \vartheta) p(\mathbf{x}_t|\mathbf{x}_{t-1}, \vartheta), \tag{6}$$

where $\vartheta = \{\mathbf{A}, \mathbf{b}, \{\mathbf{w}_l, \tilde{w}_l\}_{l=1}^L, \boldsymbol{\Sigma}_x, \mathbf{C}, \mathbf{d}, \boldsymbol{\Sigma}_y, \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0\}$ are the model parameters, and $p(\mathbf{x}_0|\vartheta) = \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$ is the initial density.

3.1. Inference with moment matching

In this section, we discuss how the posterior over the latent variables $\mathbf{x}_{0:T}$ can be inferred given data $\mathbf{y}_{1:T}$ and the model parameters ϑ. As usual for state-space models, we

do this first by building the filter density $p(\mathbf{x}_t|\mathbf{y}_{1:t}, \vartheta)$ for every time step through forward iterations, and then the smoothing density $p(\mathbf{x}_t|\mathbf{y}_{1:T}, \vartheta)$ by backward iterations. However, due to the nonlinearity in our model, we cannot solve this exactly, and we resort to an approximation procedure called moment matching (Deisenroth et al., 2009).

Forward iteration. Let us assume that we have the filter density at time step $t-1$, $p(\mathbf{x}_{t-1}|\mathbf{y}_{1:t-1}, \vartheta) = \mathcal{N}(\boldsymbol{\mu}^{\mathrm{f}}_{t-1}, \boldsymbol{\Sigma}^{\mathrm{f}}_{t-1})$ and we wish to compute the density $p(\mathbf{x}_t|\mathbf{y}_{1:t}, \vartheta)$. To do so, we first compute the prediction density:

$$p(\mathbf{x}_t|\mathbf{y}_{1:t-1}, \vartheta) = \int p(\mathbf{x}_t|\mathbf{x}_{t-1}, \vartheta)p(\mathbf{x}_{t-1}|\mathbf{y}_{1:t-1}, \vartheta)\mathrm{d}\mathbf{x}_{t-1}.$$

Due to the nonlinear mean in Eq. (2), however, this integral is not analytically solvable. Luckily, we can solve for the mean and covariance of the density above, which are given by

$$\boldsymbol{\mu}^{\mathrm{p}}_t = \mathbb{E}_{p(\mathbf{x}_{t-1}|\mathbf{y}_{1:t-1},\vartheta)}\left[\mathbf{A}\boldsymbol{\phi}(\mathbf{x}_{t-1}) + \mathbf{b}\right], \tag{7}$$

$$\boldsymbol{\Sigma}^{\mathrm{p}}_t = \boldsymbol{\Sigma}_x + \mathbb{E}_{p(\mathbf{x}_{t-1}|\mathbf{y}_{1:t-1},\vartheta)}[(\mathbf{A}\boldsymbol{\phi}(\mathbf{x}_{t-1}) + \mathbf{b})(\mathbf{A}\boldsymbol{\phi}(\mathbf{x}_{t-1}) + \mathbf{b})^\top] - \boldsymbol{\mu}^{\mathrm{p}}_t(\boldsymbol{\mu}^{\mathrm{p}}_t)^\top. \tag{8}$$

Note that for the expectations above, we need to solve the expectations $\mathbb{E}[\boldsymbol{\phi}(\mathbf{x})]$ and $\mathbb{E}\left[\boldsymbol{\phi}(\mathbf{x})\boldsymbol{\phi}(\mathbf{x})^\top\right]$ with respect to a Gaussian density. Because we chose a combination of linear and nonlinear features, where the latter have a Gaussian form with respect to $\mathbf{x}$, the expectations can be solved analytically (see Appendix A). Using the mean and covariance, we approximate the prediction density with the Gaussian density of the exact first moments. That is,

$$p(\mathbf{x}_t|\mathbf{y}_{1:t-1}, \vartheta) \approx \mathcal{N}(\boldsymbol{\mu}^{\mathrm{p}}_t, \boldsymbol{\Sigma}^{\mathrm{p}}_t). \tag{9}$$

This approximation is called moment matching. Then, we can compute the (approximate) Gaussian filter density:

$$p(\mathbf{x}_t|\mathbf{y}_{1:t}, \vartheta) = \frac{p(\mathbf{y}_t|\mathbf{x}_t, \vartheta)p(\mathbf{x}_t|\mathbf{y}_{1:t-1}, \vartheta)}{p(\mathbf{y}_t|\mathbf{y}_{1:t-1}, \vartheta)} \approx \mathcal{N}(\boldsymbol{\mu}^{\mathrm{f}}_t, \boldsymbol{\Sigma}^{\mathrm{f}}_t), \tag{10}$$

where the approximation is coming from the fact that we plug in the Gaussian approximation of the prediction density (see Appendix B). This completes the forward iteration.

Backward iteration. Next, we wish to compute the smoothing densities $p(\mathbf{x}_t, \mathbf{x}_{t+1}|\mathbf{y}_{1:T}, \vartheta)$ and $p(\mathbf{x}_t|\mathbf{y}_{1:T}, \vartheta)$. Having $p(\mathbf{x}_T|\mathbf{y}_{1:T}, \vartheta)$ from the last filtering step, we can iterate backward in time and compute

$$p(\mathbf{x}_t, \mathbf{x}_{t+1}|\mathbf{y}_{1:T}, \vartheta) = p(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{y}_{1:t}, \vartheta)p(\mathbf{x}_{t+1}|\mathbf{y}_{1:T}, \vartheta). \tag{11}$$

The last density is the smoothing density from time $t+1$, and the conditional density is given by

$$p(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{y}_{1:t}, \vartheta) = \frac{p(\mathbf{x}_{t+1}|\mathbf{x}_t, \vartheta)p(\mathbf{x}_t|\mathbf{y}_{1:t}, \vartheta)}{p(\mathbf{x}_{t+1}|\mathbf{y}_{1:t}, \vartheta)}. \tag{12}$$

However, due to the nonlinear mean in Eq. (2), this is again intractable, and we perform a Gaussian approximation via moment matching:

$$p(\mathbf{x}_t, \mathbf{x}_{t+1}|\mathbf{y}_{1:T}) \approx \mathcal{N}(\boldsymbol{\mu}^{\mathrm{s}}_{t,t+1}, \boldsymbol{\Sigma}^{\mathrm{s}}_{t,t+1}), \tag{13}$$

where we can calculate $\mathbb{E}[\mathbf{x}_t]$, $\mathbb{E}\left[\mathbf{x}_t\mathbf{x}_t^\top\right]$, and $\mathbb{E}\left[\mathbf{x}_t\mathbf{x}_{t+1}^\top\right]$ analytically with respect to the true density in Eq. (11). For explicit calculations of moments, see Appendix B.

3.2. Learning model parameters

In the previous section, we saw how to infer an approximate density over the latent variables given the model parameters ϑ. In this section, we utilize the expectation–maximization (EM) algorithm (Dempster, Laird, & Rubin, 1977; Shumway & Stoffer, 1982) to learn the model parameters. Hence, instead of maximizing the likelihood in Eq. (6) directly, we maximize the $\mathcal{Q}$ function, defined as

$$\mathcal{Q}(\vartheta, \vartheta') \doteq \mathbb{E}_\vartheta\left[\ln p(\mathbf{x}_0|\vartheta')\right] + \sum_{t=1}^{T}\left(\mathbb{E}_\vartheta\left[\ln p(\mathbf{y}_t|\mathbf{x}_t, \vartheta')\right] + \mathbb{E}_\vartheta\left[\ln p(\mathbf{x}_t|\mathbf{x}_{t-1}, \vartheta')\right]\right), \tag{14}$$

where $\mathbb{E}_\vartheta[\cdot]$ refers to the expectation by the smoothing densities. Since the observation model is linear with Gaussian noise, all its parameters $\mathbf{C}$, $\mathbf{d}$, $\boldsymbol{\Sigma}_y$ can be solved analytically. The same holds for the mean and covariance of the initial density $\boldsymbol{\mu}_0$ or $\boldsymbol{\Sigma}_0$. Though the state model is nonlinear, many parameters can be optimized analytically as well, thanks to our choice of nonlinearity. The optimal transition matrix and offset are given as follows (see Appendix C):

$$\mathbf{A}^* = \left(\sum_{t=1}^{T}\mathbb{E}\left[(\mathbf{x}_t - \mathbf{b})\boldsymbol{\phi}(\mathbf{x}_{t-1})^\top\right]\right)\left(\sum_{t=1}^{T}\mathbb{E}\left[\boldsymbol{\phi}(\mathbf{x}_{t-1})\boldsymbol{\phi}(\mathbf{x}_{t-1})^\top\right]\right)^{-1}, \tag{15}$$

$$\mathbf{b}^* = \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left[\mathbf{x}_t - \mathbf{A}\boldsymbol{\phi}(\mathbf{x}_{t-1})\right]. \tag{16}$$

Also, we can solve for the optimal covariance matrix:

$$\boldsymbol{\Sigma}_x^* = \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left[(\mathbf{x}_t - \mathbf{f}(\mathbf{x}_{t-1}))(\mathbf{x}_t - \mathbf{f}(\mathbf{x}_{t-1}))^\top\right]. \tag{17}$$

The parameters of the feature vector $\boldsymbol{\phi}(\mathbf{x})$, namely $\{\mathbf{w}_i, \tilde{w}_i\}_{i=1}^K$, are the only ones that we cannot optimize analytically. We solve them by gradient ascent, maximizing the $\mathcal{Q}$ function.

3.3. Computational considerations

Now lastly, we provide a more detailed overview of the computational advantages due to the choice of the nonlinearity by Eqs. (3) and (4). For notational convenience, we drop the time index in this subsection. For the moment matching in Section 3.1 and the computation of the $\mathcal{Q}$ function in Section 3.2, we need to compute integrals of the form

$$\int g(\mathbf{x})\phi_{\mathrm{nl},l}(\mathbf{x})p(\mathbf{x})\mathrm{d}\mathbf{x} = \int g(\mathbf{x})\exp\left(-\frac{(\mathbf{w}_l^\top\mathbf{x} - \tilde{w}_l)^2}{2}\right)p(\mathbf{x})\mathrm{d}\mathbf{x}, \tag{18}$$

where $p(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $g(\mathbf{x}) \in \{1, \mathbf{x}, \mathbf{x}\mathbf{x}^\top\}$. We note that the feature $\phi_{\mathrm{nl},l}(\mathbf{x})$ is proportional to a Gaussian, and hence

$$\exp\left(-\frac{(\mathbf{w}_l^\top \mathbf{x} - \tilde{w}_l)^2}{2}\right) p(\mathbf{x}) \propto \mathcal{N}(\boldsymbol{\mu}_\phi, \boldsymbol{\Sigma}_\phi), \tag{19}$$

where

$$\boldsymbol{\Sigma}_\phi^{-1} = \boldsymbol{\Sigma}^{-1} + \mathbf{w}_l \mathbf{w}_l^\top, \tag{20}$$

$$\boldsymbol{\mu}_\phi = \boldsymbol{\Sigma}_\phi \left[\tilde{w}_l \mathbf{w}_l + \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right]. \tag{21}$$

We see that the integral is computed with respect to a new Gaussian measure, whose precision matrix $\boldsymbol{\Sigma}_\phi^{-1}$ is a rank-one update of the matrix $\boldsymbol{\Sigma}^{-1}$. Thanks to the Sherman–Morrison formula and the matrix determinant lemma, we can compute the matrix inversion and its determinant efficiently. Both quantities are needed for solving the integral. Naturally, this also extends to integrals, where we need to integrate with respect to a Gaussian measure of the form $\phi_{\mathrm{nl},l}(\mathbf{x})\phi_{\mathrm{nl},m}(\mathbf{x})p(\mathbf{x})$, except that we need to perform a rank-one update twice or a rank-two update once.

This computational advantage is lost with the choice of a Gaussian RBF:

$$\phi_{\mathrm{nl},l}(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_l\|_2^2}{2s_l^2}\right), \tag{22}$$

where $\|\cdot\|_2$ is the Euclidian norm. With this choice, the integrals discussed above are still tractable. However, the computational convenience of the rank-one updates is gone as soon as $D_x > 1$. This implies that for each integral computation, we need to perform a full matrix inversion. Since we need to compute these integrals numerous times, this considerably slows inference and parameter optimization. Moreover, this effect increases with larger latent dimensions D_x. However, note that when $D_x = 1$, the models are equivalent; i.e., they represent the same model class and have the same computational complexity.

4. Performance evaluation

4.1. Forecasting a nonlinear oscillator

First, we apply the PNL-SS model to a noisy Van der Pol oscillator as a prototypical example of a nonlinear oscillator. The Van der Pol oscillator without noise is governed by

$$\dot{x}_1 = x_2, \tag{23}$$

$$\dot{x}_2 = \gamma(1 - x_1^2)x_2 - x_1. \tag{24}$$

We performed numerical integration of the nonlinear differential equations without adding noise to the dynamical equations. The dynamics of the nonlinear oscillator for different values of γ are shown in Fig. 2**a**. The observations of the dynamics were made at equidistant $T = 250$ points in the interval [0, 40]. For simplicity, we use discrete points to denote the time steps, i.e., $t = 1, \ldots, T$. Independent Gaussian noise with a small variance of 0.01 was added to the generated time series to obtain the observed time series.

We fitted the linear and PNL-SS models with the latent dimension $D_x = 2$ to the $D_y = 2$-dimensional observation. The example shown here used $L = 15$ kernels. To simplify the interpretation of the latent processes, we fixed $\mathbf{C} = \mathbf{I}$ and $\mathbf{d} = \mathbf{0}$ in this analysis. For comparison, we also fitted a linear state-space model (Kalman filter). The marginal likelihood of the PNL-SS model exceeds that of the linear model, as expected (Fig. 2**b**), and the difference was significant (chi-squared test, $p < 0.01$). In Fig. 2**c**, we use the learned models to estimate the underlying dynamics (left, linear model; right, PNL-SS model) from the noisy observation for $t = 1, \ldots, 125$, and forecast the dynamics for $t = 126, \ldots, 250$. The solid lines are MAP estimates, and the orange area indicates 95% credible intervals, which shows the superior performance of capturing nonlinear dynamics with the PNL-SS model. Another goal of learning the nonlinear dynamics is to analyze its vector field and nullclines to understand emergent nonlinear phenomena (Ishii & Sato, 2001). Fig. 2**d** shows the original and learned vector fields with the forecasted trajectories. While the linear model can only construct the circular vector field, the reconstructed vector fields and nullclines of the PNL-SS model faithfully capture the original ones, confirming that the learned model can forecast trajectories starting from previously unseen initial points.

4.2. Comparison with a state model using RBF kernel

The standard approach for modeling nonlinear dynamics using basis functions or Gaussian processes uses RBFs or their extensions. Both the projected kernel and RBF have the same number of parameters. The projection in each kernel Eq. (4) contains D_x parameters for $\mathbf{w}_l$ and a scalar inducing point $\tilde{w}_l$. The RBF kernel has D_x parameters for $\mathbf{c}_l$ and a scalar parameter s_l. However, as discussed above, the rank-one structure in the kernels of the PNL-SS model offers advantages in computational cost in forward inference and learning, thanks to the Sherman–Morrison formula.

Here, we investigated the computational time and goodness of fit of these two models by fitting them to the noisy Lorentz system, i.e., three-dimensional chaotic attractor dynamics, while varying the number of kernels used in the models. We fitted the PNL-SS and RBF-based model to 300 time points of Lorentz dynamics with additive Gaussian noise of variance $\sigma_{\mathrm{obs}} = 0.1$. The PNL-SS method is significantly faster than the RBF-based method, assessed by the computational time required to either complete 50 EM iterations or approach the convergence criteria (Fig. 3**a,c**). Furthermore, while having the same number of parameters, the PNL-SS model outperformed the RBF-based method in terms of the marginal likelihood, regardless of the number of kernels (Fig. 3**b,d**).

4.3. Benchmarking against other forecasting methods

We benchmarked the PNL-SS model by forecasting a univariate time series sampled from a large set of chaotic attractors with 13 other major time series prediction methods. These models vary from classic methods such as ARIMA and random forests, to deep-learning methods

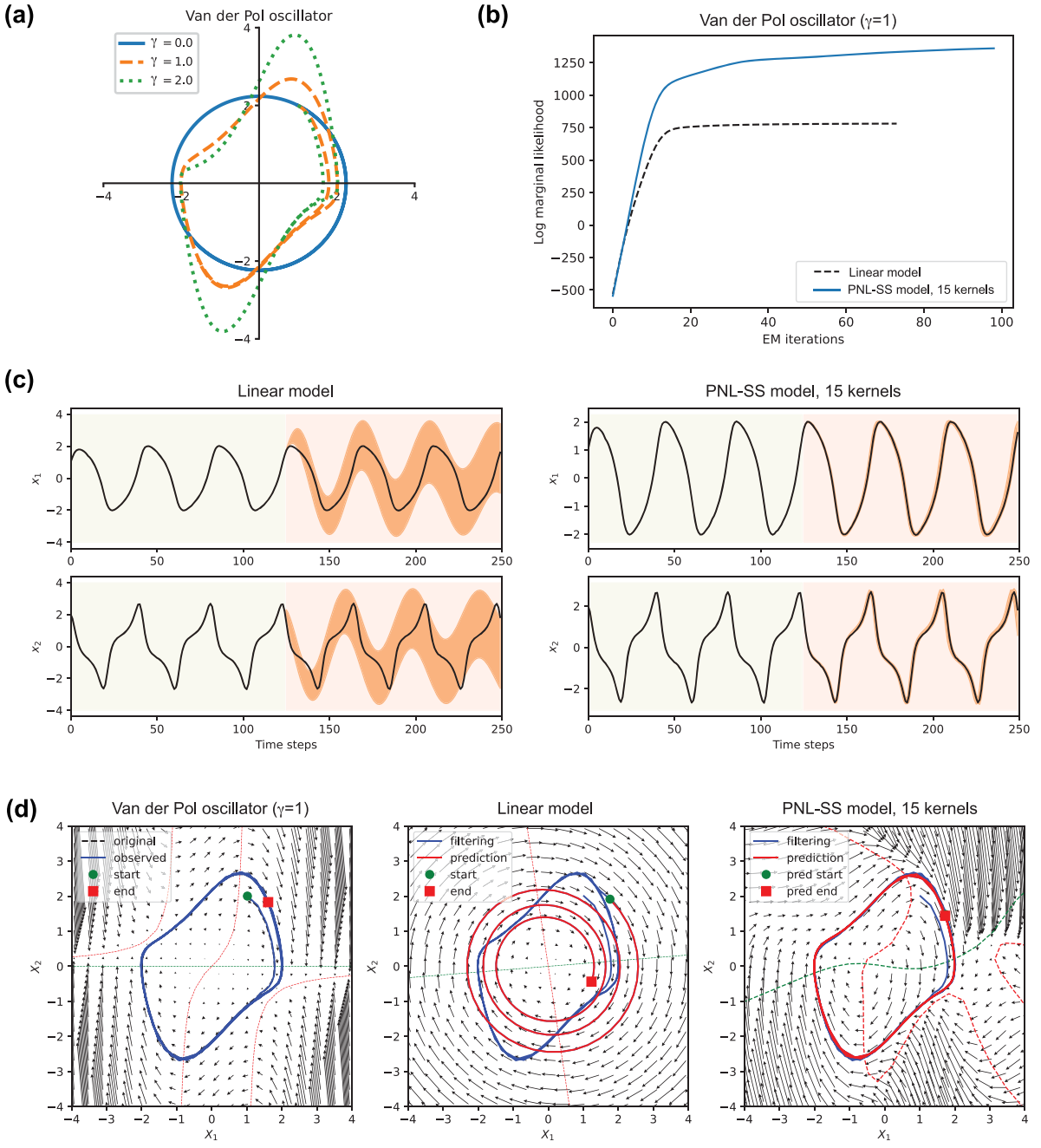


Fig. 2. Forecasting a Van der Pol oscillator with the linear and projected nonlinear state-space (PNL-SS) models. **(a)** The ground-truth dynamics of the Van der Pol oscillator with $\gamma = 1$, starting at $\mathbf{x} = [1, 2]^\top$. **(b)** Marginal likelihood of the linear (dashed) and projected nonlinear (solid) state-space models as a function of the EM iteration steps. Learning is completed when the increase of the marginal likelihood ratio from the previous value is less than 10^{-4} or at the maximum 100 iterations. **(c)** Estimation and prediction of two-dimensional latent states of the Van der Pol oscillator by the linear (left) and projected nonlinear (right) models. The data points on time $t = 1, \ldots, 125$ are learned by the models (yellow background). Smoothed estimates of the latent states are given during this period. The prediction was made for $t = 126, \ldots, 250$ (light orange background). The orange areas indicate a 95% credible interval of the prediction density. Note that the colored areas are too narrow to be visible during the learning period. **(d)** (left) The vector field (arrows) of the nonlinear oscillator in the two-dimensional space with the original (dashed black) and observed dynamics with noise (solid blue). The dashed black lines are behind the blue lines. The dotted red and green lines are nullclines, where the vector field possesses a zero component. (middle) The vector field and nullclines are learned by the linear state-space model. The blue and red solid lines are the estimated and forecasted trajectories, respectively. (right) The same as in the middle panel for the PNL-SS model. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

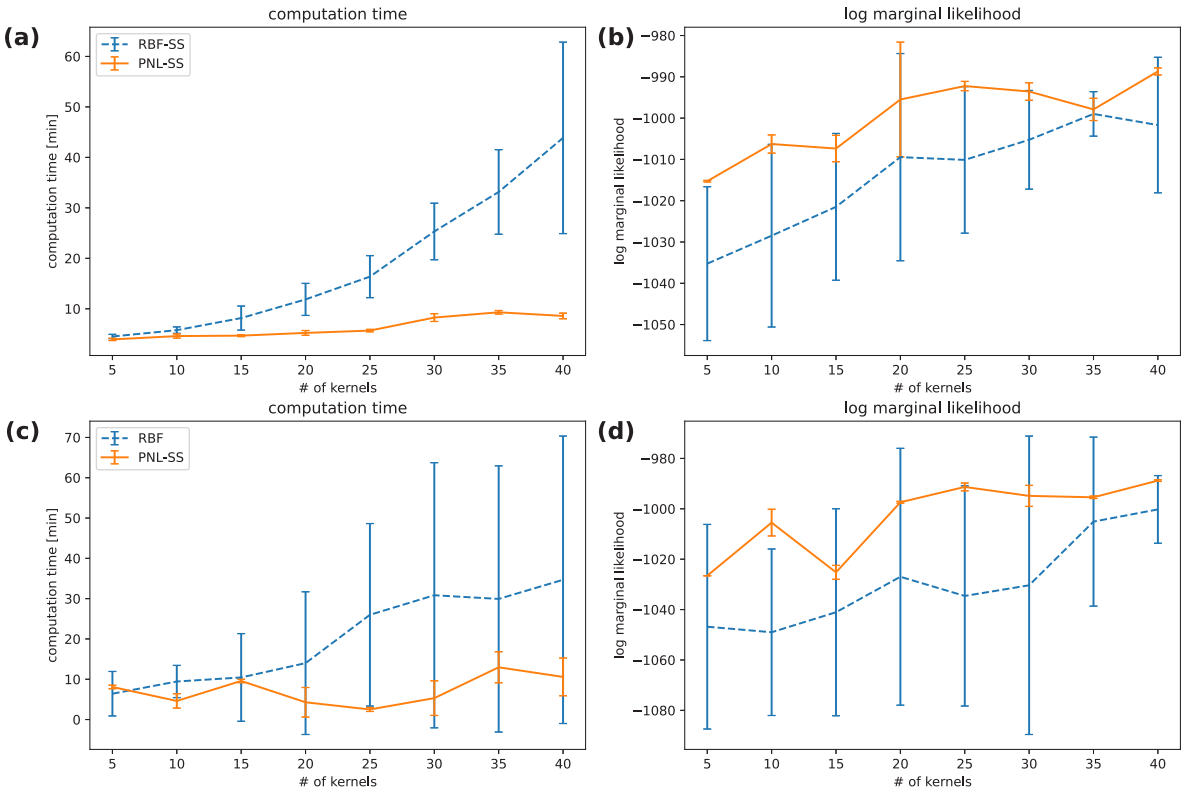


Fig. 3. Comparison of the PNL-SS and RBF-SS models in fitting a noisy Lorentz attractor. **(a)** Computational time required for completing 50 EM iterations for the PNL-SS (orange) and RBF-SS (blue) as a function of the number of kernels. The line graph for each method shows the mean for fitting 10 sample trajectories with 2SD error bars. Computation was performed with an Intel Xeon 6258R 2.7G with Tesla V100s using custom code powered by Jax. **(b)** Marginal likelihoods of the PNL-SS (orange) and RBF-SS (blue) models after completing 50 EM iterations as a function of the number of kernels. Note that the PNL-SS and RBF-SS models possess an equal number of parameters. **(c)** Computational time for each method until the same convergence criteria are satisfied. Convergence is achieved if the increase of the marginal likelihood ratio from the previous value is less than 10^{-4} or reaches the maximum EM iteration of 100. **(d)** Marginal likelihoods of each method after convergence. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

such as long short-term memory (LSTM) (Salinas et al., 2020), transformers (Kim et al., 2021; Vaswani et al., 2017), and NBEATS (Oreshkin et al., 2020). Most deep-learning methods do not treat the time series as latent dynamics in their original formulation and may be limited to forecasting a one-dimensional signal and point predictions only. In contrast, PNL-SS predictions come with uncertainty estimates at the cost of increased computational complexity of learning and prediction. Nevertheless, here we compare our probabilistic model with these competing state-of-the-art architectures.

For this goal, we employed the scheme developed by Gilpin (2021), which standardized datasets of more than 100 chaotic dynamical systems and benchmarked 13 classical and deep-learning time series prediction methods based on the Darts environment for time series prediction (Herzen et al., 2022). The dataset includes precomputed chaotic trajectories under different conditions. We selected the univariate training and test datasets that differ in the initial conditions under fine sampling granularity. See Appendix D and Gilpin (2021), Herzen et al. (2022) for further details on the dynamical systems and various other time series methods used for benchmarking. One advantage of using chaotic dynamics for benchmarking is that they are characterized by well-defined mathematical properties, such as the complexity of the trajectories, which enables us to elucidate the characteristics of each forecasting model.

To noisy observations of the 127 chaotic dynamics, we fitted 14 different time series prediction methods (including the PNL-SS method) whose hyperparameters are optimized by test datasets using the symmetric mean absolute percent error (SMAPE) as an error measure. To fit the PNL-SS model to one-dimensional time series data, we constructed delayed coordinates of D_x dimension with the same number of latent space dimensions $D_x = D_y$. By doing so, we effectively introduced dependencies within the observations in $\mathbf{y}_t$. We found that by providing this additional information to the model during training, the resulting forecasting performance improved. As additional hyperparameters, we considered the number of kernels and latent dimension from the combination of $L = [5, 10]$ and $D_x = D_y = [5, 30]$, and report the models that performed best on the validation set. See Appendix D for details.

Fig. 4**a** shows forecasts of AIZAWA (the first chaotic dynamics in the list of Gilpin's datasets) by exemplary methods, including methods based on deep learning. Note that,

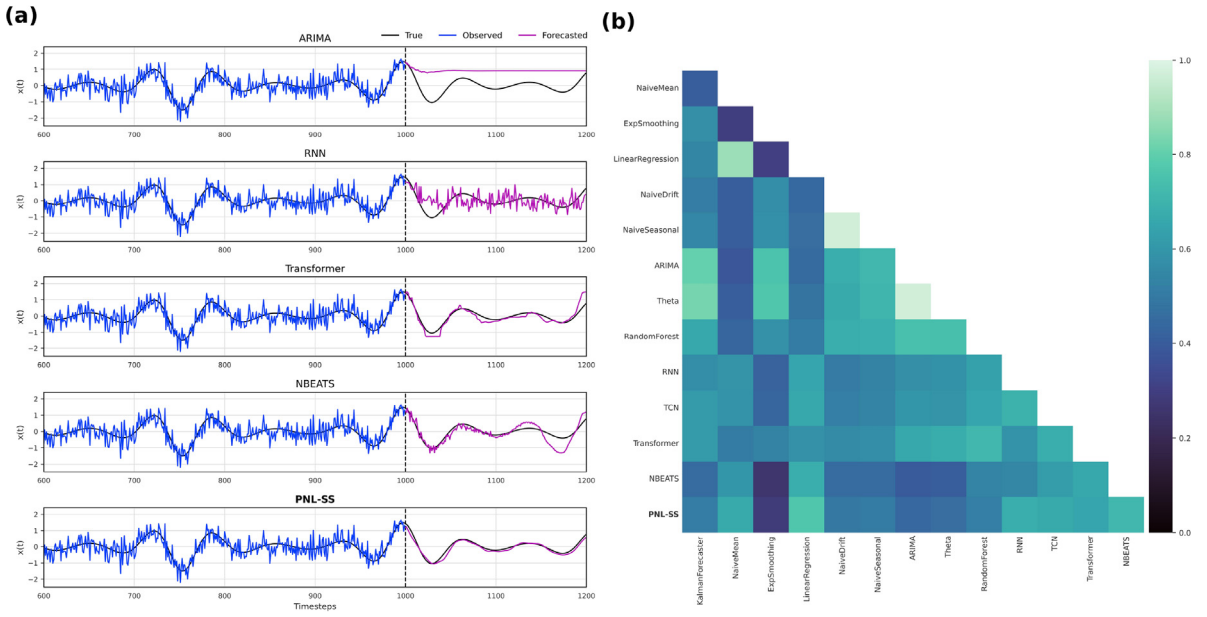


Fig. 4. Comparison of methods in forecasting noisy, chaotic time series. **(a)** Forecasting results on the AIZAWA dataset using the ARIMA, LSTM-RNN, Transformer, NBEATS, and PNL-SS methods. Models fitted to noisy observations of AIZAWA (original: smooth black lines, noisy observations: blue lines) for the first 1000 time points were used to forecast the following 200 time points (purple lines). Additive observation noise with the variance $\sigma^2_{\text{obs}} = 0.64$ was added to the original dynamics. **(b)** Spearman correlation of forecasting errors among different forecasting methods computed across all dynamical systems. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

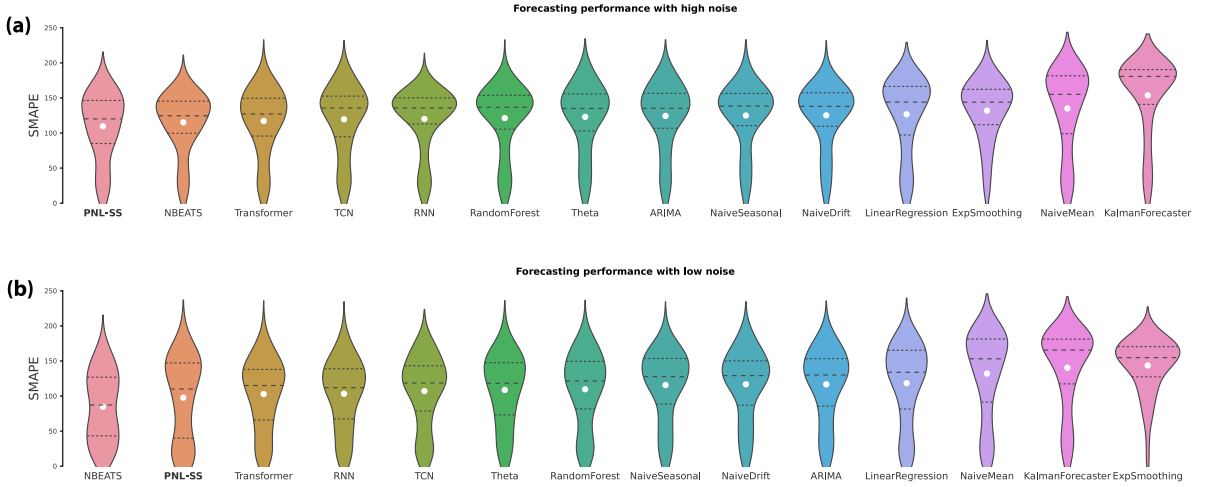


Fig. 5. Forecasting results with high and low noise for chaotic time series data. **(a)** Distributions of forecast errors for 127 dynamical systems with high observation noise across different forecasting models, sorted by increasing mean error. Independent Gaussian noise with the zero mean and variance $\sigma^2_{\text{obs}} = 0.64$ was added as the observation noise in the high noise condition. Here, the forecast error was measured by the symmetric mean absolute percentage error (SMAPE). See Appendix D for details. The white dot within each violin plot shows the mean of the SMAPEs. The dotted horizontal lines show quartiles, where the middle one is the median. **(b)** Forecasting results for low observation noise ($\sigma^2_{\text{obs}} = 0.04$). Note that the model order along the horizontal axis differs between the two panels because the relative performance of different forecasting methods changes with the levels of noise present.

for the case of AIZAWA, the optimal number of the parameters selected by LSTM-RNN, Transformer, and NBEATS were 8K, 548K, and 6.8M, respectively, whereas the PNL-SS model has only 420 parameters. Although the number of optimized parameters can differ for each dataset, the same numbers are typically chosen in other chaotic time series. Fig. 4**b** shows how the forecasting errors (SMAPE) of different models are related. The PNL-SS model (the bottom row) has a strong Spearman correlation with all the deep-learning models rather than classical methods such as exponential smoothing or ARIMA, which means that it captures the underlying complexity in the time series in a similar way as a deep-learning model does, despite being simpler in design.

Fig. 5 shows how the forecast errors (SMAPE) are distributed for 127 dynamical systems and 14 forecasting

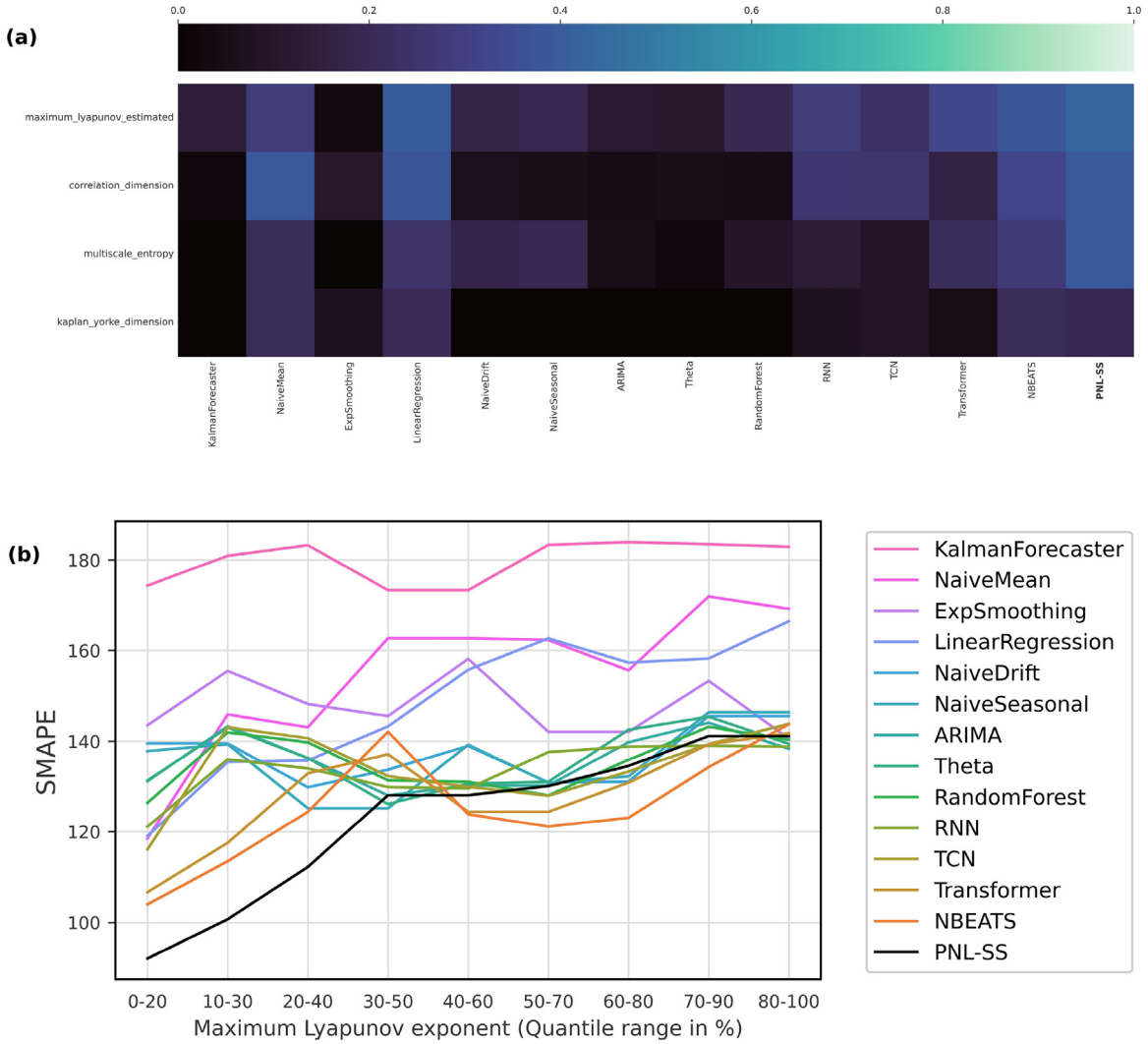


Fig. 6. Assessment of forecasting performance with respect to the properties of the chaotic systems. **(a)** Spearman correlation between forecasting errors of each forecasting method and the underlying mathematical properties (Lyapunov exponent, correlation dimension, multiscale entropy, and Kaplan–Yorke dimension) of the chaos systems. **(b)** Forecasting errors (symmetric mean absolute percent error, SMAPE) of different forecasting methods as a function of the maximum Lyapunov exponent of the dynamical systems represented in quantiles. The horizontal axis represents a sliding window that covers a 20% quantile of the distribution of the maximum Lyapunov exponent across all systems. Each point on the graph corresponds to the median performance across all dynamical systems in that quantile. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

models under high or low observation noise. We see that the PNL-SS model is one of the best models, along with the Transformer and NBEATS models. For high noise levels (Fig. 5**a**), the PNL-SS model performs better than every other method. The plots shown are sorted by mean values of the SMAPE, but sorting by median values also made the PNL-SS take first place. To see the characteristics of the forecasting methods, we examined the forecasting error of each method with respect to several standard mathematical properties that measure the degree of chaotic behavior of a system (see Gilpin (2021)). Fig. 6**a** shows the Spearman correlation between the forecasting errors and complexities of the trajectories measured by the Lyapunov exponent, correlation dimension, multi-scale entropy, and Kaplan–Yorke dimension, computed across all methods and all dynamical systems at high noise levels. Stronger positive correlations found for the PNL-SS model with these measures confirm that the performance of the model more faithfully represented the complexities of the system than the other methods. Fig. 6**b** shows the relation between the Lyapunov exponent of the system and the forecasting error. It shows that the PNL-SS performs better for the systems with low Lyapunov exponents, and that the method is competitive with deep-learning methods for forecasting complex trajectories (i.e., high Lyapunov exponents), making it the best method on average for noisy conditions (Fig. 5a).

Finally, we compared the models' ability to reconstruct dynamical systems from noisy observations. Two simple

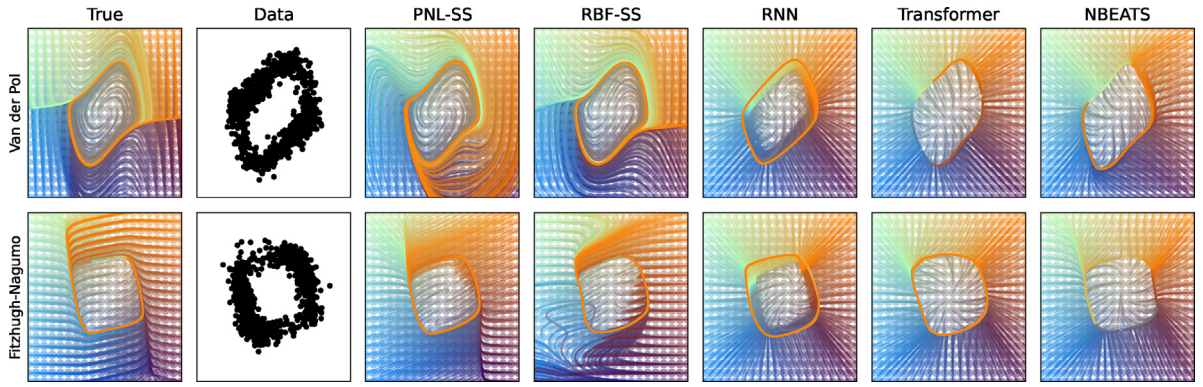


Fig. 7. Comparison of dynamical system recovery. From left to right, the underlying dynamical system, the noisy training data, and the reconstructed dynamics by PNL-SS, RBF-SS, RNN, Transformer, and NBEATS. The color of trajectories corresponds to its initial condition (colored dots). The top row is for the Van der Pol oscillator, and the bottom row is for the FitzHugh–Nagumo model. For details of the reconstruction method, see Appendix E. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

two-dimensional systems, namely the Van der Pol oscillator and a FitzHugh–Nagumo model, were learned by the PNL-SS, RBF-SS, RNN, Transformer, and NBEATS models. Note that, in contrast to the previous prediction analysis, we limited the input of each model to the most recent time point for prediction. This potentially deteriorates the predictive performance (particularly for sequence-to-sequence models like Transformer and NBEATS) but makes the dynamics in the phase space interpretable. After training the model, we predicted the dynamics for several initial conditions regularly spaced over the phase space (Fig. 7). Hence, we can assess how well the models learned the dynamics outside the regimes of the observed trajectories. For more details, see Appendix E.

The results demonstrate that all methods possess some ability to reconstruct the limit cycle. However, predictions in regimes where no data were observed varied significantly. The PNL-SS and RBF-SS models recover the dynamics of the limit cycle and reasonably extrapolate the vector fields to the previously unseen regime. Fig. E.1 provides visual intuition about how the linear and nonlinear parts of the PNL-SS model (Eq. (2)) contribute to the vector field reconstruction. In contrast, RNN, Transformer, and NBEATS fail to do so because, when starting from a point away from the limit cycle, their trajectories immediately jump to the regime of the training data. Thus, the deep-learning methods fail to extrapolate dynamics far away from the data.

5. Discussion and summary

The modeling of the nonlinear state function in our study (Eqs. (3) and (4)) is in contrast to the popular choice of the Gaussian RBF and the square exponential (SE) function of the form $\exp(-(\mathbf{x}-\mathbf{c})^\top \mathbf{\Lambda}(\mathbf{x}-\mathbf{c}))$, with $\mathbf{\Lambda}$ being a scaled identity and full-rank matrix, respectively. Our projection method can be considered a restricted version of the SE function, where $\mathbf{\Lambda}$ is a rank-one matrix, and therefore reduces the distance along the one-dimensional vector $\mathbf{w}_l$ to form a ridge function (Fig. 1**b**). We confirmed both theoretically and empirically that this choice is computationally advantageous when performing inference and learning.

In addition to the computational efficiency, the choice of the rank-one matrix in the kernel describing the network of interactions is in agreement with the low-rank structure of the dynamics ubiquitously found in complex systems (Thibeault et al., 2024). Note that the Gaussian ridge function we employed is concentrated on the projected line defined by $\mathbf{w}_l$ in the phase space but is non-concentrated in other directions (Fig. 1**c**), allowing generalization in broader regimes. However, we note that our model is a particular case of the models using the low-rank ridge functions, where we share the same kernels across the latent dimension. To explain this, consider a more general formulation in which the ith element of the nonlinear function is written as a linear combination of a bias term, a linear function, and kernel functions specific to each latent dimension:

$$f_i(\mathbf{x}) = \mathbf{A}_{\mathrm{lin},i}\boldsymbol{\phi}_{\mathrm{lin},i}(\mathbf{x}) + \mathbf{A}_{\mathrm{nl},i}\boldsymbol{\phi}_{\mathrm{nl},i}(\mathbf{x}) + b_i. \tag{25}$$

Here, the kernel vector $\boldsymbol{\phi}_{\mathrm{nl},i}(\mathbf{x}) = [\phi(h_{i,1}(\mathbf{x}))), \ldots, \phi(h_{i,L}(\mathbf{x}))]^\top$ specific to the dimension is composed of

$$h_{i,l}(\mathbf{x}) = \mathbf{w}_{i,l}^\top \mathbf{x} - \tilde{w}_{i,l}. \tag{26}$$

If $\mathbf{w}_{i,l}$ is common for all kernels (i.e., $\mathbf{w}_{i,l} = \mathbf{w}_i$ for $l = 1, \ldots, L$) and $\mathbf{A}_{\mathrm{lin},i} = \mathbf{0}$ or $\mathbf{A}_{\mathrm{lin},i} = \mathbf{w}_i$, each nonlinear function $f_i(\mathbf{x})$ is a function of a scalar value $\mathbf{w}_i^\top \mathbf{x}$. Apart from the additive noise, this state model describes one-layer continuous-valued recurrent neural network dynamics with synaptic weights $\mathbf{w}_i$ and a nonlinear activation function $f_i(\mathbf{w}_i\mathbf{x})$ ($i = 1, \ldots, D_{\mathbf{x}}$) (Valpola & Karhunen, 2002), where the function is determined in a non-parametric way rather than given as a parametric one, such as a logistic/tanh, or other types of sigmoid functions. This simplified form is still a generalization of most kernel- or GP-based state-space approaches, where the same set of kernels is used for all latent states while only their coefficients in Eq. (25) are individually optimized (Zhao & Park, 2020).

In this article, we explored the cases where the projection weights $\mathbf{w}_{i,l}$ are different for each kernel yet shared across latent variables: $\mathbf{w}_{i,l} = \mathbf{w}_l$ for all $i = 1, \ldots, D_{\mathbf{x}}$. This function can still be seen as a single-layer neural network if we consider each kernel as an activation function of a neuron (Candès, 1999; Cybenko, 1989). In this case, however, the nonlinear function $f_i(\mathbf{x})$ for each latent dimension contains a greater number of weight parameters than the case above, allowing us to capture a more detailed nonlinear transition of each latent variable, given that the number of kernels is the same. This indicates that the model potentially captures the nonlinear dynamics with fewer latent dimensions.

In summary, we provided a fast algorithm for inferring and learning nonlinear dynamics in time series data. One of the novel aspects of the proposed model is the use of kernel functions defined on projected lines to infer the nonlinear state dynamics. Our choice of the projected lines for each kernel while sharing them across latent dimensions makes the state model more expressive at capturing the dynamics with fewer dimensions while keeping the complexity of the model reasonably low. We showcased the model's ability to capture nonlinear dynamics and make accurate predictions using a prototypical model of noisy nonlinear oscillators and demonstrated its merits in terms of computational efficiency and goodness of fit over the conventional RBF-based method. Finally, our exhaustive survey on forecasting 127 noisy, chaotic time series revealed that our method outperforms deep-learning methods, given the small samples and high noise, speaking to the alternative approach for forecasting time series.

Code availability The python code for the projected nonlinear state-space model is available in the Github repository,

github.com/christiando/timeseries_models.

The code uses the toolbox for operating a Gaussian distribution available in the Github repository,

github.com/christiando/gaussian-toolbox.

Reproducibility was not assessed by the editorial office due to high computational costs, but all code and data for figure generation are available for verification at

github.com/christiando/pnlss.

CRediT authorship contribution statement

Christian Donner: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Anuj Mishra:** Writing – review & editing, Writing – original draft, Visualization, Validation, Investigation, Data curation. **Hideaki Shimazaki:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Resources, Project administration, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

CD is supported by a Swiss Data Science Center project grant (C19-06). HS was supported by New Energy and Industrial Technology Development Organization (NEDO), Japan and JSPS KAKENHI Grant Number JP 20K11709, 21H05246, 24K21518.

Appendix A. Solving Gaussian integrals

In this section, we give the explicit solutions expectations over functions involving the nonlinear kernels $\phi_{\mathrm{nl},l}(\mathbf{x})$ with respect to a generic Gaussian $p(\mathbf{x})$ with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$.

First, the expectation is given as

$$\mathbb{E}_{p(\mathbf{x})}\left[\phi_{\mathrm{nl},l}(\mathbf{x})\right] = \frac{\sqrt{|\tilde{\boldsymbol{\Sigma}}_l|}\exp\left(\frac{1}{2}\tilde{\boldsymbol{\mu}}_l^\top \tilde{\boldsymbol{\Sigma}}_l^{-1}\tilde{\boldsymbol{\mu}}_l\right)}{\sqrt{|\boldsymbol{\Sigma}|}\exp\left(\frac{\tilde{w}_l^2}{2} + \frac{1}{2}\boldsymbol{\mu}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)} \tag{A.1}$$

with $\tilde{\boldsymbol{\Sigma}}_l = \left[\boldsymbol{\Sigma}^{-1} + \mathbf{w}_l\mathbf{w}_l^\top\right]^{-1}$ and $\tilde{\boldsymbol{\mu}}_l = \tilde{\boldsymbol{\Sigma}}_l\left[\tilde{w}_l\mathbf{w}_l + \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right]$. The second integral we encounter is of the following form:

$$\mathbb{E}_{p(\mathbf{x})}\left[\mathbf{x}\phi_{\mathrm{nl},l}(\mathbf{x})\right] = \tilde{\boldsymbol{\mu}}_l\frac{\sqrt{|\tilde{\boldsymbol{\Sigma}}_l|}\exp\left(\frac{1}{2}\tilde{\boldsymbol{\mu}}_l^\top \tilde{\boldsymbol{\Sigma}}_l^{-1}\tilde{\boldsymbol{\mu}}_l\right)}{\sqrt{|\boldsymbol{\Sigma}|}\exp\left(\frac{\tilde{w}_l^2}{2} + \frac{1}{2}\boldsymbol{\mu}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)}. \tag{A.2}$$

The last integral we need to solve is over the product of two kernels:

$$\mathbb{E}_{p(\mathbf{x})}\left[\phi_{\mathrm{nl},l}(\mathbf{x})\phi_{\mathrm{nl},m}(\mathbf{x})\right] = \frac{\sqrt{|\tilde{\boldsymbol{\Sigma}}_{lm}|}\exp\left(\frac{1}{2}\tilde{\boldsymbol{\mu}}_{lm}^\top \tilde{\boldsymbol{\Sigma}}_{lm}^{-1}\tilde{\boldsymbol{\mu}}_{lm}\right)}{\sqrt{|\boldsymbol{\Sigma}|}\exp\left(\frac{\tilde{w}_l^2+\tilde{w}_m^2}{2} + \frac{1}{2}\boldsymbol{\mu}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right)} \tag{A.3}$$

with $\tilde{\boldsymbol{\Sigma}}_{lm}^{-1} = \left[\mathbf{w}_l\mathbf{w}_l^\top + \mathbf{w}_m\mathbf{w}_m^\top + \boldsymbol{\Sigma}^{-1}\right]$ and $\tilde{\boldsymbol{\mu}}_{lm} = \tilde{\boldsymbol{\Sigma}}_{lm}\left[\tilde{w}_l\mathbf{w}_l + \tilde{w}_m\mathbf{w}_m + \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}\right]$. With these integrals, we can compute all the moments we need for learning the proposed algorithm.

Appendix B. Details on filtering and smoothing procedure

In this section, we provide the mean and covariances for predicting, filtering, and smoothing densities that we obtain by moment matching. The moments for the prediction density are straightforward to calculate with (7)–(8) and the results in Appendix A. Since we use a linear observation model, we get the standard Kalman filter result for the mean and covariance of the filtering density. That is,

$$\boldsymbol{\Sigma}_t^{\mathrm{f}} = \boldsymbol{\Sigma}_t^{\mathrm{p}} - \boldsymbol{\Sigma}_t^{\mathrm{p}}\mathbf{C}^\top\boldsymbol{\Sigma}_y^{-1}\mathbf{C}\boldsymbol{\Sigma}_t^{\mathrm{p}}, \tag{B.1}$$

$$\boldsymbol{\mu}_t^{\mathrm{f}} = \boldsymbol{\mu}_t^{\mathrm{p}} + \boldsymbol{\Sigma}_t^{\mathrm{p}}\mathbf{C}^\top\boldsymbol{\Sigma}_y^{-1}(\mathbf{y}_t - \mathbf{C}\boldsymbol{\mu}_t^{\mathrm{p}} - \mathbf{d}). \tag{B.2}$$

Once we iterate forward until $t = T$, we obtain the last smoothing density $p(\mathbf{x}_T|\mathbf{y}_{1:T})$, which is equal to the filtering density. To get the smoothing density for all the previous time steps, we perform the following backward iteration. Assuming we have $p(\mathbf{x}_{t+1}|\mathbf{y}_{1:T})$, we want

$p(\mathbf{x}_t|\mathbf{y}_{1:T})$. To derive this density, we first need to calculate the lag-one density:

$$p(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{y}_{1:t}) = \frac{p(\mathbf{x}_{t+1}|\mathbf{x}_t)p(\mathbf{x}_t|\mathbf{y}_{1:t})}{p(\mathbf{x}_{t+1}|\mathbf{y}_{1:t})}. \tag{B.3}$$

Because the numerator involves the nonlinear state density, we again resort to moment matching and get

$$p(\mathbf{x}_{t+1}, \mathbf{x}_t|\mathbf{y}_{1:t}) \approx \mathcal{N}\left(\begin{pmatrix}\boldsymbol{\mu}^{\mathrm{p}}_{t+1}\\ \boldsymbol{\mu}^{\mathrm{f}}_{t}\end{pmatrix}, \begin{pmatrix}\boldsymbol{\Sigma}^{\mathrm{p}}_{t+1} & \mathbf{M}_{t+1,t}\\ \mathbf{M}^{\top}_{t+1,t} & \boldsymbol{\Sigma}^{\mathrm{f}}_{t}\end{pmatrix}\right). \tag{B.4}$$

For the covariances, we get

$$\begin{aligned}\mathbf{M}_{t+1,t} &= \mathrm{Cov}(\mathbf{x}_{t+1}, \mathbf{x}_t)\\ &= \mathbb{E}_{p(\mathbf{x}_t|\mathbf{y}_{1:t})}\left[(\mathbf{A}\boldsymbol{\phi}(\mathbf{x}_t) + \mathbf{b})\mathbf{x}_t^{\top}\right] - \boldsymbol{\mu}^{\mathrm{p}}_{t+1}(\boldsymbol{\mu}^{\mathrm{f}}_{t})^{\top},\end{aligned} \tag{B.5}$$

which involves only integrals solved in Appendix A. With this, we approximate the lag-one density by

$$\begin{aligned}p(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{y}_{1:t}) \approx& \mathcal{N}(\boldsymbol{\mu}^{\mathrm{f}}_{t} + \mathbf{M}^{\top}_{t+1,t}(\boldsymbol{\Sigma}^{\mathrm{p}}_{t+1})^{-1}(\mathbf{x}_{t+1} - \boldsymbol{\mu}^{\mathrm{p}}_{t+1}),\\ &\boldsymbol{\Sigma}^{\mathrm{f}}_{t} - \mathbf{M}^{\top}_{t+1,t}(\boldsymbol{\Sigma}^{\mathrm{p}}_{t+1})^{-1}\mathbf{M}_{t+1,t}) \qquad \text{(B.6)}\\ =& \mathcal{N}(\mathbf{M}^{\mathrm{l}}_{t|t+1}\mathbf{x}_{t+1} + \mathbf{b}^{\mathrm{l}}_{t|t+1}, \boldsymbol{\Sigma}^{\mathrm{l}}_{t|t+1}) \qquad \text{(B.7)}\end{aligned}$$

with

$$\mathbf{M}^{\mathrm{l}}_{t|t+1} = \mathbf{M}^{\top}_{t+1,t}(\boldsymbol{\Sigma}^{\mathrm{p}}_{t+1})^{-1}, \tag{B.8}$$

$$\mathbf{b}^{\mathrm{l}}_{t|t+1} = \boldsymbol{\mu}^{\mathrm{f}}_{t} - \mathbf{M}^{\mathrm{l}}_{t|t+1}\boldsymbol{\mu}^{\mathrm{p}}_{t+1}. \tag{B.9}$$

With this, we get the approximation

$$\begin{aligned}p(\mathbf{x}_t, \mathbf{x}_{t+1}|\mathbf{y}_{1:T}) &= p(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{y}_{1:t})p(\mathbf{x}_{t+1}|\mathbf{y}_{1:T})\\ &\approx \mathcal{N}(\boldsymbol{\mu}^{\mathrm{s}}_{t,t+1}, \boldsymbol{\Sigma}^{\mathrm{s}}_{t,t+1}).\end{aligned} \tag{B.10}$$

From this, we can also straightforwardly get the marginal $p(\mathbf{x}_t|\mathbf{y}_{1:T}) \approx \mathcal{N}(\boldsymbol{\mu}^{\mathrm{s}}_{t}, \boldsymbol{\Sigma}^{\mathrm{s}}_{t})$.

Appendix C. Details on calculating the Q-function

To optimize the Q-function in Eq. (14), we again need to solve some Gaussian integrals. Since the initial density and the observation model are Gaussian, the first and second terms are straightforward to calculate. The last term involves the nonlinear state model (2). Hence, for time t, we get

$$\begin{aligned}&\mathbb{E}_{\vartheta}\left[\ln p(\mathbf{x}_t|\mathbf{x}_{t-1}, \vartheta')\right] =\\ &\mathbb{E}_{\vartheta}\left[-\frac{1}{2}\left(\mathbf{x}_t - \mathbf{A}\boldsymbol{\phi}(\mathbf{x}_{t-1}) - \mathbf{b}\right)^{\top}\boldsymbol{\Sigma}_x^{-1}\left(\mathbf{x}_t - \mathbf{A}\boldsymbol{\phi}(\mathbf{x}_{t-1}) - \mathbf{b}\right)\right]\\ &- \frac{1}{2}\ln|2\pi\boldsymbol{\Sigma}_x|.\end{aligned} \tag{C.1}$$

Again, we see that this factorizes into terms that can be solved by the integrals discussed in Appendix A. Also, it is straightforward to obtain the optimal $\mathbf{A}$, $\mathbf{b}$ and $\boldsymbol{\Sigma}_x$ from this, by setting the gradients to zero. For the kernel parameters $\{\mathbf{w}_l, \tilde{w}_l\}_{l=1}^{L}$, we find them numerically by the quasi-Newton method (L-BFGS-B) and automatic differentiation.

Appendix D. Details on forecasting chaotic dynamics

The benchmark of the PNL-SS model with other methods is based on the framework and code developed by Gilpin (2021). This framework provides standardized datasets of over 100 chaotic dynamical systems and codes for benchmarking classical and deep-learning time series prediction methods. The benchmarking is built on the Darts environment, a Python library for time series analysis (Herzen et al., 2022). We implemented our PNL model as one of the forecasting methods callable by Darts.

While we followed the procedures and parameter settings described by Gilpin (2021), we briefly summarize them below. Each one-dimensional chaotic time series is composed of 1200 time points, of which the first 1000 points are used for training the model parameters and the last 200 points are used for testing the prediction. We added white Gaussian noise with zero mean and a standard deviation of 0.8 for high noise conditions and 0.2 for low noise conditions to the training data. The prediction error compared to the original data was computed using the symmetric mean absolute percentage error (SMAPE), defined as $200\langle|z_t - \hat{z}_t| / (|z_t| + |\hat{z}_t|)\rangle$, where z_t denotes the test data, $\hat{z}_t$ the forecasting values, and $\langle\cdot\rangle$ the time average within a forecasting period (i.e., $t = 1001, \ldots, 1200$). The hyperparameters of the models, such as the input and output chunk length or the number of network layers for deep-learning models, were optimized by the grid search function of Darts by applying the same procedure above to datasets of the chaotic dynamics obtained separately from the evaluation procedure above. The ranges of the hyperparameters to be searched were unchanged from Gilpin (2021). This hyperparameter search was performed independently for each chaotic system under high and low noise conditions. In the analysis, we removed the chaos systems of GenesioTesi, Hadley, SprottD, and StickSlipOscillator, due to errors encountered during ARIMA fitting.

For the PNL-SS model, we embedded the time series data into the D_y-dimensional delayed coordinates and used them to train the model. A vector $\mathbf{y}_t$ of the embedded trajectories at time point t is composed of D_y elements, each of which is a value shifted $200/D_y$ points in time, while the last element is the original value at t. We used the same number of dimensions as the latent dimension ($D_x = D_y$). The last element in the vector was used for the prediction. For each chaos trajectory, the embedding dimension D_y and the number of kernels L are hyperparameters that we determined by a grid search from the combination of $D_y = 5$ or 10 and $L = 5$ or 30.

Appendix E. Details on 2D dynamical system comparison

To create Fig. 7, we simulated data from two 2D systems. The first is the Van der Pol oscillator (Eqs. (23) and (24)) with $\gamma = 1$. The second is a FitzHugh–Nagumo system with the governing equations

$$\dot{x}_1 = 4(x_1 - x_1^3/3 - x_2 + 0.7), \tag{E.1}$$

$$\dot{x}_2 = 4(x_1 + 0.8 - x_2)/12.5. \tag{E.2}$$

Both systems exhibit oscillatory behavior. The second system can be seen as an example of a "stiff" system, where

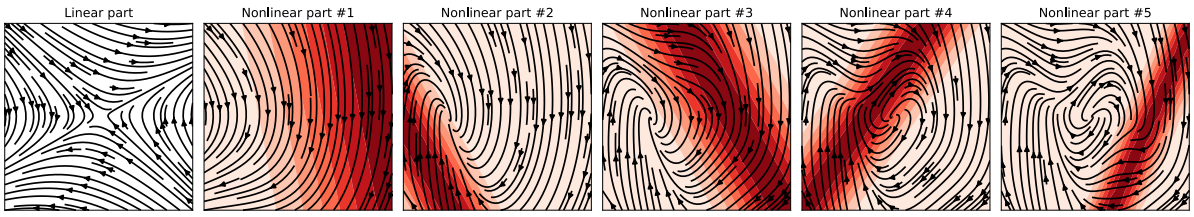


Fig. E.1. PNL-SS vector field reconstruction. From left to right: The streamlines in each panel depict the reconstruction of the vector field by the sum of the linear function and up to the lth nonlinear kernel function. The order of the kernels is according to the norm of their weight vectors $\|\mathbf{A}_{\text{nl},l}\|$ $(l = 1, \ldots, L)$. The last panel shows the fully learned vector field. The red contour in each panel indicates the newly added kernel function $\phi_{\text{nl},l}(\mathbf{x})$.

$\dot{x}_1 \gg \dot{x}_2$ or $\dot{x}_2 \gg \dot{x}_1$, depending on the state of the system. These types of systems can be problematic for data-driven learning of dynamics (Holt, Qian, & van der Schaar, 2022). For both systems, we created a dataset where we ran the dynamics with the initial condition $\mathbf{x}(0) = [0, 2]^\top$ by solving the equations numerically for the time points $t = 0, 0.1, 0.2, \ldots, 119.8, 119.9$, resulting in 1200 time steps. Then, we standardized each dimension and added Gaussian noise with 0.2 standard deviation. This corresponds to the low noise condition in Fig. 5. The first 1000 data points of each dataset were then used for training. The last 200 points were used only to evaluate the early-stopping criterion for the deep-learning methods.

In contrast to Fig. 5, we restricted all methods' inputs to the most recent time point of the data, since otherwise it was difficult to extract an interpretable phase space. The latent space of the Linear, RBF-SS, and PNL-SS models was set to $D_x = 2$. For the latter two methods, we set $L = 5$, as in Fig. 2. To have an RNN model that is as close as possible to the PNL-SS, it was fit with one layer with five hidden units (similar to the number of kernels in PNL-SS). For the NBEATS and Transformer models, we used the default settings of the Darts library. All models were trained until convergence or until they reached 200 epochs. Fig. E.1 illustrates how each of the learned kernels of the PNL-SS model contributes to the reconstruction of the vector field of the Van der Pol oscillator.

References

Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. In *International conference on machine learning*. arXiv:1409.0473.

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, *5*(2), 157–166.

Candès, E. J. (1999). Harmonic analysis of neural networks. *Applied and Computational Harmonic Analysis*, *6*(2), 197–218.

Chen, Z., et al. (2003). Bayesian filtering: From Kalman filters to particle filters and beyond. *Statistics*, *182*(1), 1–69.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, *2*(4), 303–314.

d'Ascoli, S., Becker, S., Mathis, A., Schwaller, P., & Kilbertus, N. (2023). ODEFormer: Symbolic regression of dynamical systems with transformers. arXiv preprint arXiv:2310.05573.

Deisenroth, M. P., Huber, M. F., & Hanebeck, U. D. (2009). Analytic moment-based Gaussian process filtering. In *Proceedings of the 26th annual international conference on machine learning* (pp. 225–232).

Deisenroth, M. P., & Mohamed, S. (2012). Expectation propagation in Gaussian process dynamical systems. In *Advances in neural information processing systems*: *vol. 25*, (pp. 2609–2617).

Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society. Series B. Statistical Methodology*, *39*(1), 1–22.

Duncker, L., Bohner, G., Boussard, J., & Sahani, M. (2019). Learning interpretable continuous-time models of latent stochastic dynamical systems. In *International conference on machine learning* (pp. 1726–1734).

Durbin, J., & Koopman, S. J. (2012). *Time series analysis by state space methods*. Oxford University Press.

Frigola, R., Chen, Y., & Rasmussen, C. E. (2014). Variational Gaussian process state-space models. In *Advances in neural information processing systems*: *vol. 27*, (pp. 3680–3688).

Frigola, R., Lindsten, F., Schön, T. B., & Rasmussen, C. E. (2013). Bayesian inference and learning in Gaussian process state-space models with particle MCMC. In *Advances in neural information processing systems*: *vol. 26*, (pp. 3156–3164).

Ghahramani, Z., & Roweis, S. T. (1999). Learning nonlinear dynamical systems using an EM algorithm. In *Advances in neural information processing systems*: *vol. 11*, (pp. 431–437).

Gilpin, W. (2021). Chaos as an interpretable benchmark for forecasting and data-driven modelling. In *Advances in neural information processing systems*: *vol. 35*, arXiv:2110.05266.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).

Herzen, J., Lässig, F., Piazzetta, S. G., Neuer, T., Tafti, L., Raille, G., et al. (2022). Darts: User-friendly modern machine learning for time series. *Journal of Machine Learning Research*, *23*(1), 1–6, 24.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*, 1735–1780, 11.

Holt, S. I., Qian, Z., & van der Schaar, M. (2022). Neural Laplace: Learning diverse classes of differential equations in the Laplace domain. In *International conference on machine learning* (pp. 8811–8832). PMLR.

Honkela, A., & Valpola, H. (2004). Unsupervised variational Bayesian learning of nonlinear models. *vol. 17*, In *Advances in neural information processing systems* (pp. 593–600).

Ishii, S., & Sato, M.-A. (2001). Reconstruction of chaotic dynamics by on-line EM algorithm. *Neural Networks*, *14*(9), 1239–1256.

Julier, S. J., & Uhlmann, J. K. (2004). Unscented filtering and nonlinear estimation. *Proceedings of the IEEE*, *92*(3), 401–422.

Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Trans. ASME, Ser. D, J. Basic Eng.*, *82*, 34–45.

Kim, T., Kim, J., Tae, Y., Park, C., Choi, J.-H., & Choo, J. (2021). Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International conference on learning representations*.

Ko, J., & Fox, D. (2011). Learning GP-BayesFilters via Gaussian process latent variable models. *Autonomous Robots*, *30*(1), 3–23.

Ko, J., Klein, D. J., Fox, D., & Haehnel, D. (2007). GP-UKF: Unscented Kalman filters with Gaussian process prediction and observation models. In *2007 IEEE/RSJ international conference on intelligent robots and systems* (pp. 1901–1907).

Krishnan, R. G., Shalit, U., & Sontag, D. (2015). Deep Kalman filters. arXiv preprint arXiv:1511.05121.

Krishnan, R., Shalit, U., & Sontag, D. (2017). Structured inference networks for nonlinear state space models. *Proceedings of the AAAI Conference on Artificial Intelligence*, *31*(1).

Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., et al. (2023). iTransformer: Inverted transformers are effective for time series forecasting. arXiv preprint arXiv:2310.06625.

Oreshkin, B. N., Carpov, D., Chapados, N., & Bengio, Y. (2020). N-beats: Neural basis expansion analysis for interpretable time series forecasting. In *Proceedings of the international conference on learning representations*. arXiv:1905.10437.

Pearlmutter, B. A. (1989). Learning state space trajectories in recurrent neural networks. *Neural Computation*, *1*(2), 263–269.

Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). Deepar: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, *36*(3), 1181–1191.

Seshadri, P., Yuchi, S., & Parks, G. T. (2019). Dimension reduction via Gaussian ridge functions. *SIAM/ASA Journal on Uncertainty Quantification*, *7*(4), 1301–1322.

Shumway, R. H., & Stoffer, D. S. (1982). An approach to time series smoothing and forecasting using the EM algorithm. *Journal of Time Series Analysis*, *3*(4), 253–264.

Thibeault, V., Allard, A., & Desrosiers, P. (2024). The low-rank hypothesis of complex systems. *Nature Physics*, *20*, 294–302.

Turner, R., Deisenroth, M., & Rasmussen, C. (2010). State-space inference and learning with Gaussian processes. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 868–875).

Valpola, H., & Karhunen, J. (2002). An unsupervised ensemble learning method for nonlinear dynamic state-space models. *Neural Computation*, *14*(11), 2647–2692.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*: *vol. 30*, (pp. 5998–6008).

Wang, J. M., Fleet, D. J., & Hertzmann, A. (2005). Gaussian process dynamical models. In *Advances in neural information processing systems*: *vol. 18*, (pp. 929–936).

Wang, J. M., Fleet, D. J., & Hertzmann, A. (2007). Gaussian process dynamical models for human motion. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *30*(2), 283–298.

Zhao, Y., & Park, I. M. (2020). Variational online learning of neural dynamics. *Frontiers in Computational Neuroscience*, *14*, http://dx.doi.org/10.3389/fncom.2020.00071.